JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class InflaterInputStream

java.lang.Object
  java.io.InputStream
    java.io.FilterInputStream
      java.util.zip.InflaterInputStream

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

Direct Known Subclasses:

`GZIPInputStream`, `ZipInputStream`

```
public class InflaterInputStream
extends FilterInputStream
```

This class implements a stream filter for uncompressing data in the "deflate" compression format. It is also used as the basis for other decompression filters, such as GZIPInputStream.

Unless otherwise noted, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

Decompressor Usage

An `InflaterInputStream` created without specifying a decompressor will create a decompressor at construction time, and close the decompressor when the input stream is closed.

If a decompressor is specified when creating a `InflaterInputStream`, it is the responsibility of the caller to close the decompressor after closing the input stream.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the `try`-with-resources statement.

Since:

1.1

See Also:

`Inflater`

Field Summary

Fields

Modifier and Type	Field	Description

protected byte[]	**buf**	Input buffer for decompression.
protected Inflater	**inf**	Decompressor for this stream.
protected int	**len**	The total number of bytes read into the input buffer.

Fields declared in class FilterInputStream

in

Constructor Summary

Constructors

Constructor	Description
InflaterInputStream(InputStream in)	Creates a new input stream and decompressor with a default buffer size.
InflaterInputStream(InputStream in, Inflater inf)	Creates a new input stream with the specified decompressor and a default buffer size.
InflaterInputStream(InputStream in, Inflater inf, int size)	Creates a new input stream with the specified decompressor and buffer size.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
int	**available**()	Returns 0 after EOF has been reached, otherwise always return 1.
void	**close**()	Closes this input stream and releases any system

		resources associated with the stream.
protected void	**fill**()	Fills input buffer with more data to decompress.
void	**mark**(int readlimit)	Marks the current position in this input stream.
boolean	**markSupported**()	Tests if this input stream supports the `mark` and `reset` methods.
int	**read**()	Reads a byte of uncompressed data.
int	**read**(byte[] b, int off, int len)	Reads uncompressed data into an array of bytes, returning the number of inflated bytes.
void	**reset**()	Repositions this stream to the position at the time the `mark` method was last called on this input stream.
long	**skip**(long n)	Skips specified number of bytes of uncompressed data.

Methods declared in class FilterInputStream

read

Methods declared in class InputStream

nullInputStream, readAllBytes, readNBytes, readNBytes, skipNBytes, transferTo

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

inf

```
protected Inflater inf
```

Decompressor for this stream.

buf

```
protected byte[] buf
```

Input buffer for decompression.

len

```
protected int len
```

The total number of bytes read into the input buffer.

Constructor Details

InflaterInputStream

```
public InflaterInputStream(InputStream in,
                           Inflater inf,
                           int size)
```

Creates a new input stream with the specified decompressor and buffer size.

Closing this input stream will not close the given decompressor.

Parameters:

`in` - the input stream

`inf` - the decompressor ("inflater")

`size` - the input buffer size

Throws:

`IllegalArgumentException` - if `size <= 0`

InflaterInputStream

```
public InflaterInputStream(InputStream in,
                           Inflater inf)
```

Creates a new input stream with the specified decompressor and a default buffer size.

Closing this input stream will not close the given decompressor.

Parameters:

`in` - the input stream

`inf` - the decompressor ("inflater")

InflaterInputStream

```
public InflaterInputStream(InputStream in)
```

Creates a new input stream and decompressor with a default buffer size.

The decompressor will be closed when this input stream is closed.

Parameters:

`in` - the input stream

Method Details

read

```
public int read()
         throws IOException
```

Reads a byte of uncompressed data. This method will block until enough input is available for decompression.

Overrides:

`read` in class `FilterInputStream`

Returns:

the byte read, or -1 if end of compressed input is reached

Throws:

`IOException` - if an I/O error has occurred

See Also:

`FilterInputStream.in`

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads uncompressed data into an array of bytes, returning the number of inflated bytes. If `len` is not zero, the method will block until some input can be decompressed; otherwise, no bytes are read and `0` is returned.

If this method returns a nonzero integer *n* then `buf[off]` through `buf[off+`*n*`-1]` contain the uncompressed data. The content of elements `buf[off+`*n*`]` through `buf[off+`*len*`-1]` is undefined, contrary to the specification of the `InputStream` superclass, so an implementation is free to modify these elements during the inflate operation. If this method returns `-1` or throws an exception then the content of `buf[off]` through `buf[off+`*len* `-1]` is undefined.

Overrides:

`read` in class `FilterInputStream`

Parameters:

`b` - the buffer into which the data is read

`off` - the start offset in the destination array `b`

`len` - the maximum number of bytes read

Returns:

the actual number of bytes inflated, or -1 if the end of the compressed input is reached or a preset dictionary is needed

Throws:

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

See Also:

`FilterInputStream.in`

available

```
public int available()
             throws IOException
```

Returns 0 after EOF has been reached, otherwise always return 1.

Programs should not count on this method to return the actual number of bytes that could be read without blocking.

Overrides:

`available` in class `FilterInputStream`

Returns:

1 before EOF and 0 after EOF.

Throws:

`IOException` - if an I/O error occurs.

skip

```
public long skip(long n)
          throws IOException
```

Skips specified number of bytes of uncompressed data. This method may block until the specified number of bytes are skipped or end of stream is reached.

Overrides:

`skip` in class `FilterInputStream`

Implementation Note:

This method skips at most `Integer.MAX_VALUE` bytes.

Parameters:

`n` - the number of bytes to skip. If `n` is zero then no bytes are skipped.

Returns:

the actual number of bytes skipped, which might be zero

Throws:

`IOException` - if an I/O error occurs or if this stream is already closed

`IllegalArgumentException` - if `n < 0`

See Also:

`InputStream.skipNBytes(long)`

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `FilterInputStream`

Throws:

`IOException` - if an I/O error has occurred

See Also:

`FilterInputStream.in`

fill

```
protected void fill()
             throws IOException
```

Fills input buffer with more data to decompress.

Implementation Requirements:

This method will read up to `buf`.length bytes into the input buffer, `buf`, starting at element `0`. The `len` field will be set to the number of bytes read.

Throws:

`IOException` - if an I/O error has occurred

`EOFException` - if the end of input stream has been reached unexpectedly

markSupported

```
public boolean markSupported()
```

Tests if this input stream supports the `mark` and `reset` methods. The `markSupported` method of `InflaterInputStream` returns `false`.

Overrides:

`markSupported` in class `FilterInputStream`

Returns:

a `boolean` indicating if this stream type supports the `mark` and `reset` methods.

See Also:

`InputStream.mark(int)`, `InputStream.reset()`

mark

```
public void mark(int readlimit)
```

Marks the current position in this input stream.

Overrides:

`mark` in class `FilterInputStream`

Implementation Requirements:

The `mark` method of `InflaterInputStream` does nothing.

Parameters:

`readlimit` - the maximum limit of bytes that can be read before the mark position becomes invalid.

See Also:

`InputStream.reset()`

reset

```
public void reset()
           throws IOException
```

Repositions this stream to the position at the time the `mark` method was last called on this input stream.

Overrides:

`reset` in class `FilterInputStream`

Implementation Requirements:

The method `reset` for class `InflaterInputStream` does nothing except throw an `IOException`.

Throws:

`IOException` - if this method is invoked.

See Also:

`InputStream.mark(int)`, `IOException`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.